

VirtualVoxel: Real-Time Large Scale Scene Visualization and Modification

Jinyuan Yang
Computer Science
University College Dublin
Dublin, Ireland
jinyuan.yang@ucdconnect.ie

Abraham G. Campbell
Computer Science
University College Dublin
Dublin, Ireland
abey.campbell@ucd.ie

ABSTRACT

This short paper introduces VirtualVoxel, a novel rasterization-based voxel renderer that enables real-time rendering of $4M^3$ resolution voxel scenes. VirtualVoxel combines the benefits of virtual texture and virtual geometry, allowing for efficient storage and rendering of texture and geometry information simultaneously.

Similar to the VirtualTexture approach, VirtualVoxel streams the appropriate level of detail into GPU memory, ensuring optimal performance. However, unlike previous graph-based voxel visualization methods, VirtualVoxel stores data linearly and employs the rasterization rendering pipeline, resulting in highly efficient modification of large scenes.

VirtualVoxel's high-performance capabilities make it ideal for a wide range of applications, from video games to scientific visualizations. It can render scenes at resolutions of up to $4M^3$ voxels in real-time, providing artists and designers with a powerful new tool for creating stunning graphics.

CCS CONCEPTS

• Computing methodologies → Rasterization.

KEYWORDS

software rasterization, voxel rendering, real-time rendering

ACM Reference Format:

Jinyuan Yang and Abraham G. Campbell. 2023. VirtualVoxel: Real-Time Large Scale Scene Visualization and Modification. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Posters (SIGGRAPH '23 Posters)*, August 06–10, 2023, Los Angeles, CA, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3588028.3603664>

1 INTRODUCTION

Rendering dense voxel grids in real-time can be challenging, leading researchers to develop methods such as Sparse Voxel Octree (SVO) [Laine and Karras 2010] and HashDag [Careil et al. 2020] which use graph-based structures to store sparse voxel representations. However, these methods require sophisticated compression techniques to fit the scene data into GPU memory, and editing the structures in real-time can be difficult as they contain static data and require pre-generation.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH '23 Posters, August 06–10, 2023, Los Angeles, CA, USA

© 2023 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0152-8/23/08.

<https://doi.org/10.1145/3588028.3603664>

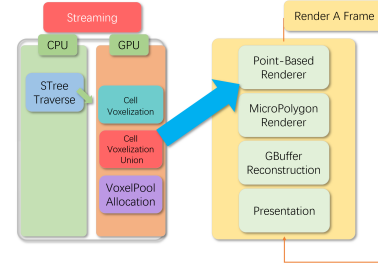


Figure 1: The workflow of the two important parts of VirtualVoxel.

Motivated by the need for real-time visualization and modification of high-resolution voxel scenes, we present VirtualVoxel. Our method builds on the success of VirtualTexture and streams only the necessary level of detail into GPU memory, enabling real-time rendering of voxel scenes up to $4M^3$ resolution.

Unlike other methods, VirtualVoxel does not require pre-generation of voxel data but instead leverages real-time voxelization [Crassin and Green 2012] to convert the scene into voxel representation on the fly. This approach simplifies the visualization and editing of large-scale scenes, making it possible for artists and designers to work with voxel-based graphics. We believe that our method is an advancement in voxel rendering technology and opens up new possibilities for real-time visualization and editing of high-resolution voxel scenes.

2 METHOD OVERVIEW

As shown in Fig.1, the VirtualVoxel renderer is made with two important parts, streaming and rendering. In the following sections, we introduce how these two parts function to render a large-scale scene.

2.1 Data Structure

VirtualVoxel scenes employ a two-level structure for streaming voxel data. The high-level structure consists of a 2D uniform grid, with each grid chunk containing a low-level sparse octree. We refer to this Octree as the STree, as it is responsible for streaming. Distinct from SVO [Laine and Karras 2010], each STree node holds a $64 \times 64 \times 64$ resolution voxel data. In subsequent sections, we use the term "cell" to denote an STree node containing voxel data.

On the GPU, a voxel pool is utilized to store the data of each cell. When a new cell is streamed to the GPU, the voxel pool allocates a new page for the cell, with each page accommodating $16,384$ (128×128) voxels.

2.2 Streaming

During rendering, the CPU traverses the STree to find the suitable level of details and streams the cell to GPU. We use a simple distance-based metric to measure LOD error. If the LOD error is above zero, the CPU evaluates the child cells, as the current cell does not meet the LOD metric.

$$LODError = \|CellCenterPos - CamPos\| - (CellSize \times StreamingFactor)$$

The STree is pre-generated using a method similar to SVO. The scene is voxelized and the STree is constructed hierarchically. For each cell of STree, we only record whether it exists, allowing us to efficiently store and access sparse voxel data.

When traversing the STree, the renderer evaluates each cell from the root cell. Whenever a suitable LOD is found or this cell is a leaf node, a cell-building request is sent to GPU.

On the GPU, we use a voxel pool to store the data of each cell. When the GPU handles the cell-building request, it first voxelizes the cell in 64^3 resolution following the method [Crassin and Green 2012] and allocates a page for this cell in the voxel pool. The result is written into a 64^3 size buffer. Next, the voxelization result is atomically appended to the allocated page.

2.3 Render a Frame

For each frame, the voxels are rendered as points to a 64-bit visibility buffer [Burns and Hunt 2013]. The layout of the visibility buffer is $depth_int < 32 | cell_offset < 12 | voxel_offset$. Each compute thread renders 64 voxels of a cell. When rendering each cell, frustum culling is used to cull any invisible cells. Then in the next pass, the renderer reconstructs the voxel from the visibility buffer. Differing from the last pass, we use a method similar to Nanite [Karis et al. 2021] to render voxels. As demonstrated in Nanite [Karis et al. 2021], the micro polygon renderer is very fast at rendering pixel-scale triangles. Each compute thread reads one pixel from the visibility buffer to reconstruct the voxel as a geometry cube. Then the voxel is rendered as a cube with 12 triangles. The result contains position and albedo. The rendering result is written to a 64-bit framebuffer with AtomicMin operation. The layout of the framebuffer is $depth_int < 32 | albedo$.

2.4 Real-Time Modification

In our example application, we support two types of modifications: Union and Subtraction. The Union operation adds a cube to the scene, while the Subtraction operation removes a cube from it. When a user selects a position P on the screen using a mouse click, a screen-pick operation retrieves the depth value from the framebuffer at P. Subsequently, the world position wP of P is determined using the inverted camera matrix. To modify the scene, we must update the STree cells surrounding wP. To achieve this, we compute an Axis-Aligned Bounding Box (AABB) and update all intersecting cells.

During the update process, a voxelization shader is executed to re-voxelize the cell. A buffer containing the Subtraction operation is utilized to apply the operation. This buffer stores a list of subtraction cube positions and scales. During voxelization, the voxelizer eliminates voxels from cells by verifying whether a voxel resides within the subtraction cube. Following this pass, a compute shader

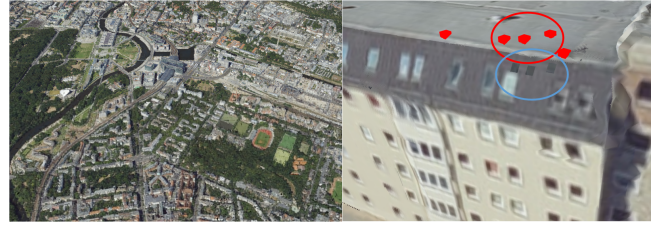


Figure 2: Left: a figure to show the test scene. Right: a figure to show the scene modification. The red circle demonstrates union operation and the blue circle represents subtraction operation.

for the Union operation is executed. Unlike the first voxelization pass, it takes the union cube as input. The Union voxelization result is then merged with the regular voxelization result. Finally, the combined voxelization result is atomically appended to the allocated voxel page.

3 RESULT AND CONCLUSION

To evaluate the performance of VirtualVoxel, we made a test scene using the Berlin Data Set ¹. This dataset contains multiple city regions. In this test, we treat each region as a chunk. Our test scene contains 16×16 chunks, each chunk is made by a 12-level STree. Therefore, the resolution of this test scene is $4M^3$. The Streaming-Factor of the LOD error is set as 8 to stream the city data.

The example renderer runs on a Windows 10 desktop with AMD Threadripper 2950X and RTX4090. The average frame time is 6.0459 ms under 4K resolution and the result is measured by the camera fly-through of the test scene.

In conclusion, we have demonstrated the feasibility of employing software rasterization to render large-scale voxel scenes with efficient and straightforward real-time editing capabilities. However, this approach is not without limitations. The current design supports only the rendering of opaque objects. Due to the atomic max operation's inability to blend color values, transparent rendering remains unachievable. Despite these constraints, our method offers a promising foundation for the development of advanced real-time voxel rendering solutions.

REFERENCES

- Christopher A Burns and Warren A Hunt. 2013. The visibility buffer: a cache-friendly approach to deferred shading. *Journal of Computer Graphics Techniques (JCGT)* 2, 2 (2013), 55–69.
- Victor Careil, Markus Billeter, and Elmar Eisemann. 2020. Interactively modifying compressed sparse voxel representations. In *Computer Graphics Forum*, Vol. 39. Wiley Online Library, 111–119.
- Cyril Crassin and Simon Green. 2012. Octree-based sparse voxelization using the GPU hardware rasterizer. *OpenGL Insights* (2012), 303–318.
- Brian Karis, Rune Stubbe, and Graham Wihlidal. 2021. A Deep Dive into Nanite Virtualized Geometry. In *ACM SIGGRAPH*.
- Samuli Laine and Tero Karras. 2010. Efficient sparse voxel octrees. In *Proceedings of the 2010 ACM SIGGRAPH symposium on Interactive 3D Graphics and Games*. 55–63.

¹<https://www.businesslocationcenter.de/downloadportal>